

L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765



MECHTRON 3TB4
Lab Tutorial 3 Report
Lab 3 Prelab Report

Date: March 6, 2026

Lab Tutorial 3 Report

Overview

The goal of this tutorial is to build and implement a Finite Impulse Response (FIR) digital filter in MATLAB to decrypt secret information. This is done by removing the sinusoidal noise components within the audio signal. Each sound file is unique to each group that contains a secret code with some noise added in different frequencies in the audio file, making the code cryptic. This process is achieved by performing frequency analysis using Fast Fourier Transform (FFT), identifying the noise frequency, designing an FIR filter to eliminate noise, verifying the filter performance using frequency response plots, and recovering the secret code from the filtered audio.

Part 1: Building a Digital Filter using MATLAB

The Digital Filter was created in MATLAB using the DSP system toolboxes' functions to decrypt the secret code in the sound file L01_Group6.wav.

The audio file is first inputted "y" for the sampled data, and as "fs" for the sampled rate. This resulted in a sampling frequency "fs" of 8000 Hz.

To determine the noise frequency, an FFT was performed on the original sound. Firstly, the FFT length was chosen using " $N_{FFT} = 2^{\text{nextpow2}(L)}$ ", which represents the number of points used in the FFT. This is because the FFT is a divide-and-conquer algorithm and is much faster when the length is a power of 2, allowing the computational complexity to be $O(N \log N)$, which is why it was chosen. The 2^{nextpow2} ensures the N_{FFT} is close to the length of the original signal, which is greater than or equal to the signal length "L." Using N_{FFT} , a single-sided amplitude spectrum plot was created to determine the noise frequency. As shown in the figure below, f_{noise} was observed at 2000 Hz.

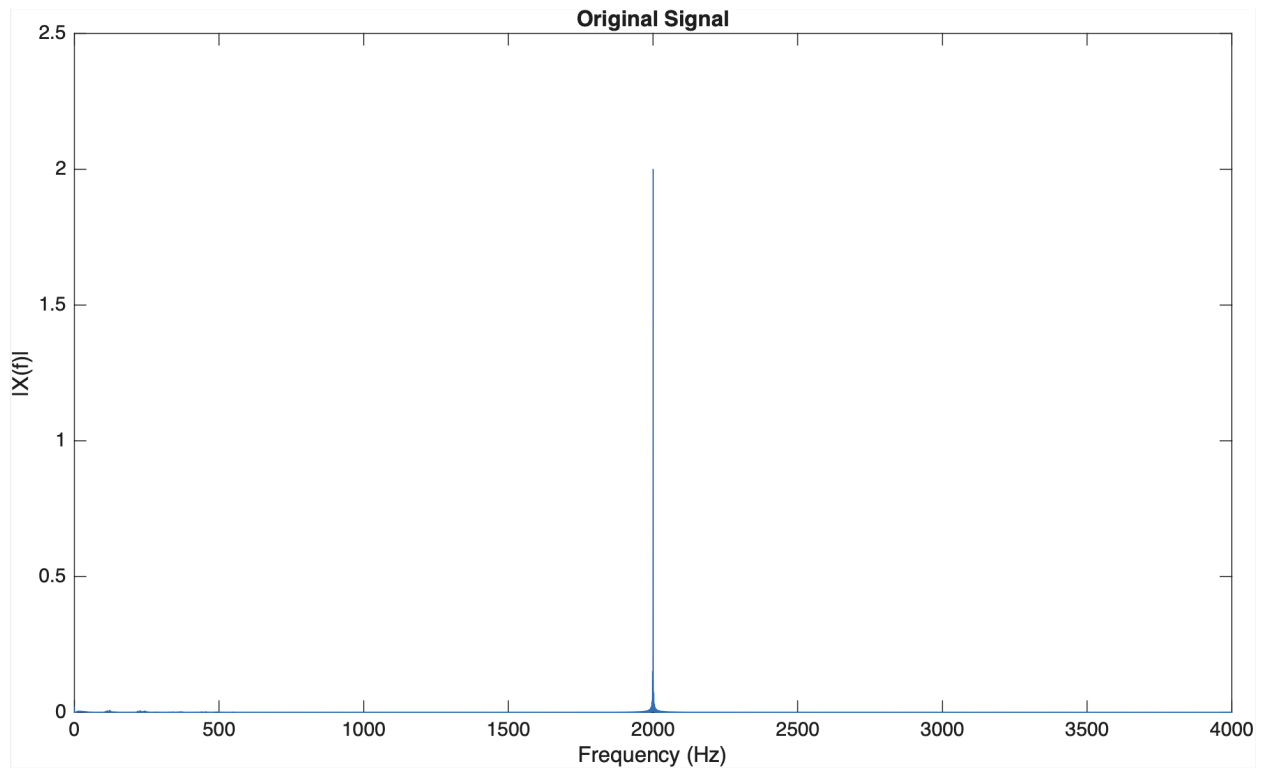


Figure 1. Single-sided amplitude plot of the original signal from L01_Group6.wav

The FIR filter requires the frequency to be normalized with respect to $f_s/2$, resulting in " $f_{kill} = f_{noise} / (f_s/2)$ " that is used for the filtered design. This is due to the MATLAB FIR function "firgr" requiring normalized frequencies rather than in units of Hz. The reason it is normalized with respect to $f_s/2$ is the Nyquist frequency, which is the highest frequency that can be represented after sampling, since $0 \leq f \leq f_s/2$, which is the same as $0 \leq f_{normalized} \leq 1$. The resulting $f_{kill} = 2000 / (8000/2) = 0.5$. Through testing, it was confirmed that the filter_length of 64 removed the frequency. The filter is then plotted as the frequency response of the filter to verify it satisfies the requirements using `freqz(coeff,1)`. The coefficients were scaled to 32768 for hardware implementation since it is equivalent to 2^{15} for the signed 16-bit used in the FPGA. The coefficients were saved in coefficients.txt for FIR hardware modules.

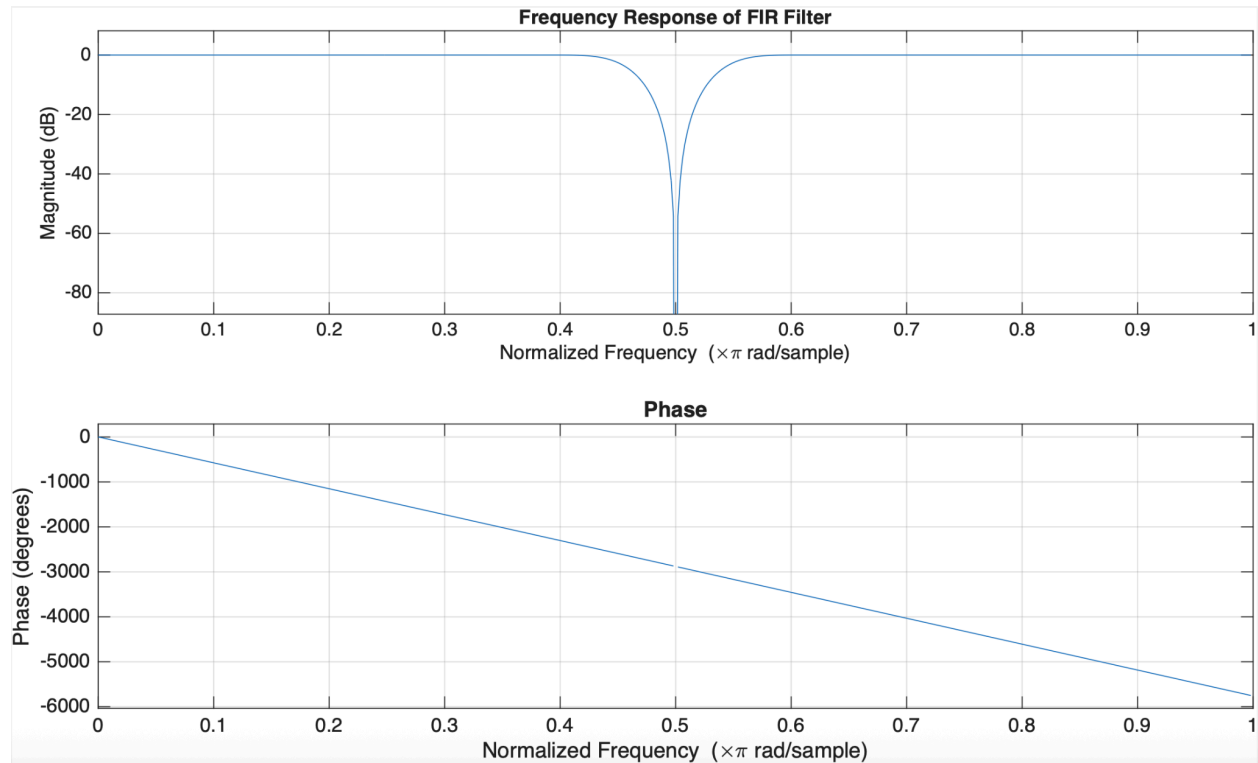


Figure 2. Frequency response of the FIR filter (top) and the phase (bottom) vs the normalized frequency.

The signal was filtered using `filtfilt`, which ensures the filtered audio maintains the original code while removing the noise. FFT is then applied again onto the filtered signal to check if the noise is removed at 2000 Hz. The unfiltered audio file had a very high pitch, distorting the secret code. The filtered audio file had the secret code **JTAC03** for L01_Group6.wav. Another plot showing the FFT of the original signal vs the filtered signal was used to determine the absence of the noise at 2000 Hz.

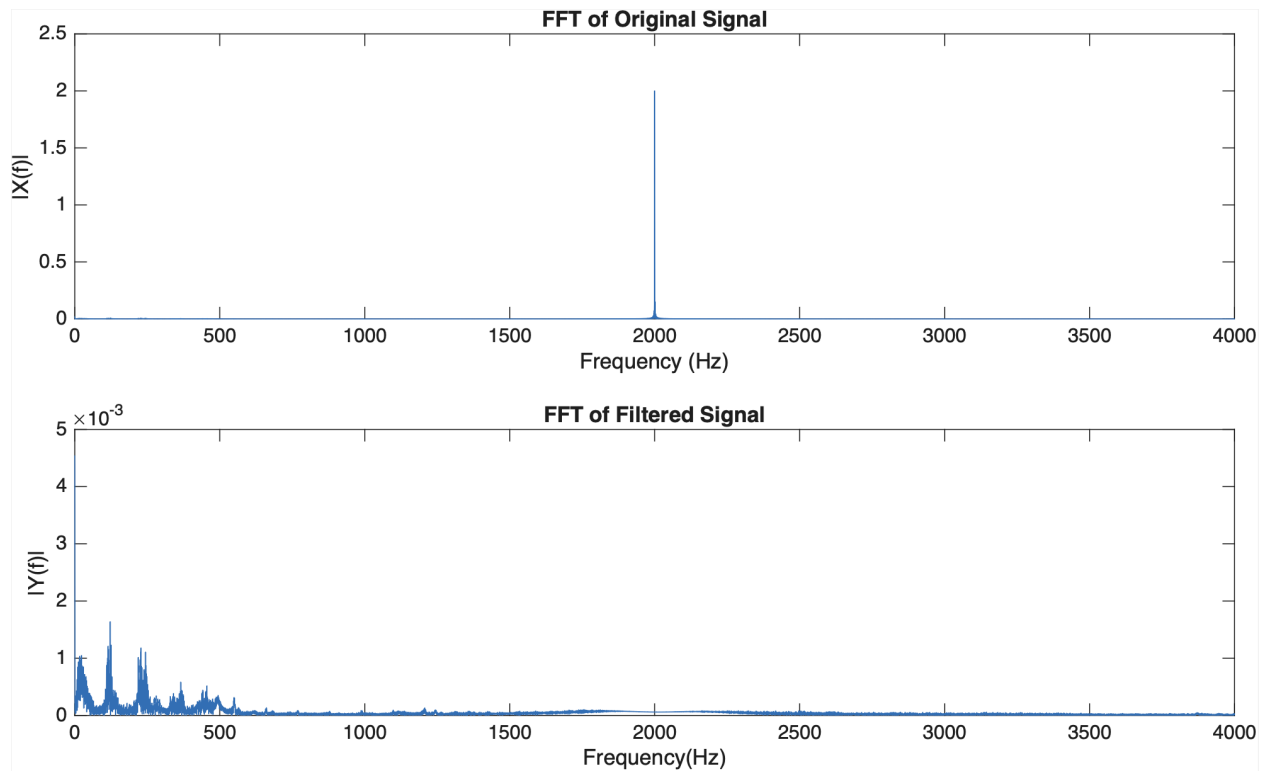


Figure 3. Comparing the first plot showing the FFT of the original signal from L01_Group6.wav (above), to the second plot showing the FFT of the original signal from L01_Group6.wav (below).

Lab 3 Prelab Report

Overview

In this prelab, two simple Digital-Signal-Processing (DSP) are created and implemented on the DE1-SoC board which are the FIR filter (SW[0] enabled) and an echo machine (SW[1] enabled). They are both used to process the given L01_Group6.wav file to decrypt the secret message masked by a high frequency pitch at 2000 Hz, determined using the Fast-Fourier Transform (FFT) function in MATLAB in Tutorial 3. The two DSP modules are integrated with a 3-to-1 multiplexer to select between the different signal processing paths.

The FIR filter uses coefficients obtained from the coefficients.txt file generated by MATLAB in Tutorial 3 and the delay element was set based on the 2 last digits of the student number as explained in Part 1. In MATLAB, each of the coefficients are scaled by 2^{15} to convert the floating-point values into fixed-point representation suitable for FPGA implementation. The multiplier.v module (generated from multiplier.qip) handles the multiplication of the 16-bit input samples and the scaled coefficients.

The echo machine utilizes the shiftregister.v module (generated from shiftregister.qip) that creates a delay with the number of taps determined by the last two digits of the student number as described in Part 1. This delay produces a delayed version of the audio signal that is combined with the original input signal to generate an echo effect.

The modules are integrated with the dsp_subsystem.v module and connected to the provided top-level module lab3.v. This module allows for modular development and easy debugging of the individual components while enabling real-time audio processing on the FPGA.

Part 1: Samples in the Sound

The sampling frequency provided by the sound file is 8 kHz. The number of samples in the sound of the L01_Group6.wav was derived from its lasting X milliseconds, where X is the last two digits of the student number. X represents the delayed samples and is used to calculate for the total samples. As shown below, each delayed sample has 8 samples. This will be later implemented where X is the number_of_taps and 8 is the tap_distance in shiftregister.v.

$$\text{Number of Samples} = 8000 \text{ samples/second} \times X \text{ seconds} / 1000 \text{ milliseconds} = 8X \text{ samples}$$

For student number 65:

$$\text{Number of Samples for student \# 65} = 8 \times 65 = 520 \text{ samples.}$$

For student number 07:

$$\text{Number of Samples for student \# 07} = 8 \times 7 = 56 \text{ samples.}$$

Part 2: 3-to-1 Multiplexer

The 3-to-1 Multiplexer is used to switch between generating an audio of the original signal, the signal after the FIR filter is applied, or an echo effect using the echo machine. A minimum of 2-bit input was required for switching between 3 different cases. The original sound is set at 2'b00, the FIR filter is set at 2'b01, and the echo machine is set at 2'b10 with the default set as the original signal.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Mar 4 21:09:56 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	mux3_1
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	10 / 32,070 (< 1 %)
Total registers	0
Total pins	66 / 457 (14 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 4. Screenshot of the 3-to-1 Multiplexer module compilation report.

Table 1. Summary for Components for the 3-to-1 Multiplexer Module in PreLab 3.

Feature	Quantity
Total number of logic elements	10
Total number of registers	0
Total number of pins	66
Max number of logic elements that can fit on FPGA	32,070

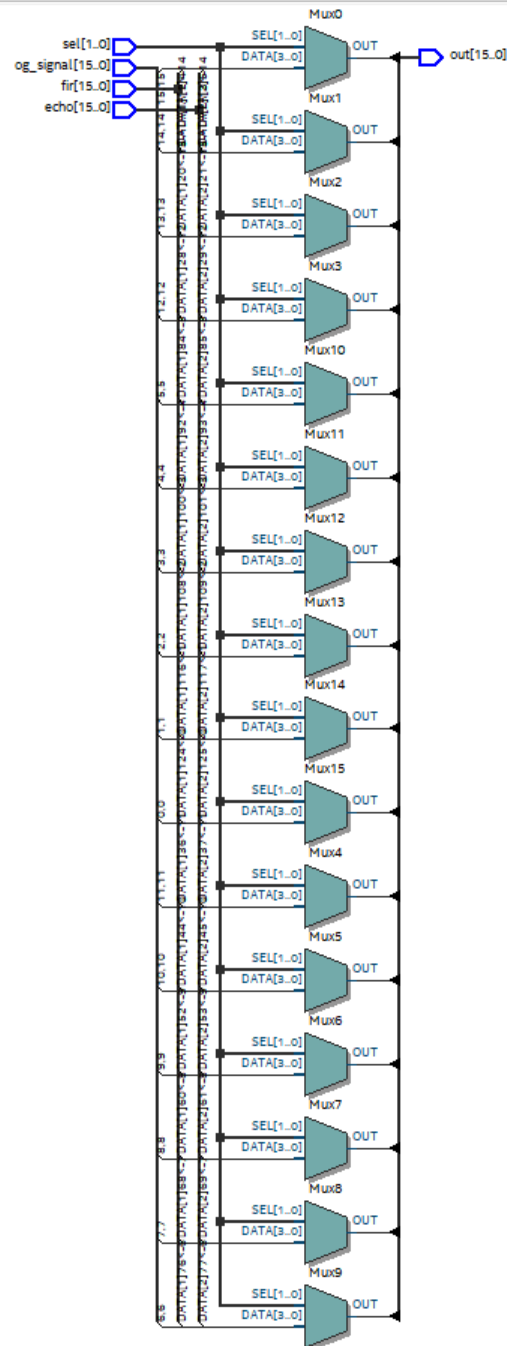


Figure 5. Screenshot of the implemented 3-to-1 Multiplexer Module.

```
1 module mux3_1(input signed [15:0] og_signal, fir, echo,  
2               input [1:0] sel,  
3               output reg signed [15:0] out);  
4  
5     always @(*) begin  
6       case (sel)  
7         2'b00: out = og_signal;  
8         2'b01: out = fir;  
9         2'b10: out = echo;  
10        default: out = og_signal;  
11      endcase  
12    end  
13 endmodule // mux3_1
```

Figure 6. Screenshot of the module of the 3-to-1 Multiplexer Module.

Part 3: Parameterizable FIR filter

The FIR filter module processes the audio samples using a finite impulse response filtering structure. The samples are stored in `x_delay` implemented as a shift register to preserve the current and previous inputs values. Each delayed sample is multiplied by the corresponding filter coefficient generated from the `coefficients.txt` file from MATLAB and scaled for fixed-point representation. A for loop adder is used to sum all the products together, initialized at 32-bits to ensure there is no overflow. The sum is later on scaled to fit within 16 bits by “>>> 15” operation. The final register is output per clock cycle when reset is not 0. The total coefficients are set to 65 as well based on the MATLAB parameter `filter_len = 64`. The parameter can be adjusted for any variable below 65.

Flow Summary	
 <<Filter>>	
Flow Status	Successful - Wed Mar 4 21:17:54 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	fir_filter
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	257 / 32,070 (< 1 %)
Total registers	1141
Total pins	34 / 457 (7 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	64 / 87 (74 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 7. Screenshot of the Parameterizable FIR filter module compilation report.

Table 2. Summary for Components for the Parameterizable FIR filter Module in PreLab 3.

Feature	Quantity
Total number of logic elements	257
Total number of registers	1141
Total number of pins	34
Max number of logic elements that can fit on FPGA	32,070

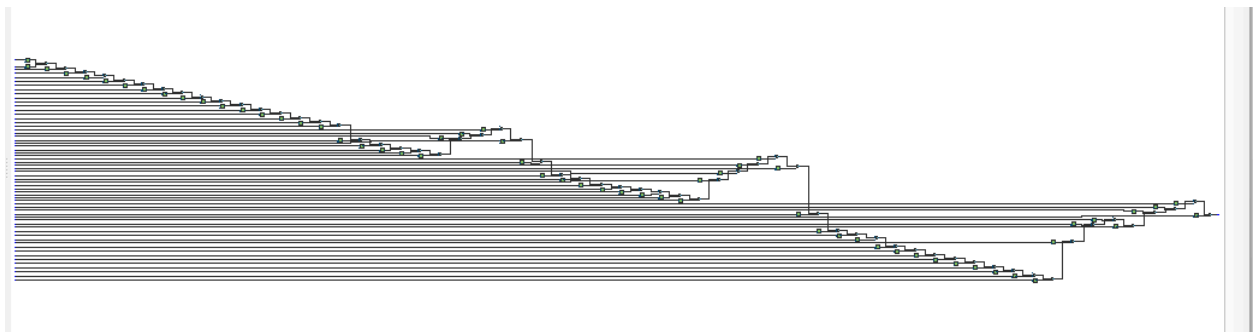


Figure 8. Screenshot of the implemented Parameterizable FIR filter Module.

```

1 module fir_filter(
2     input wire clk,           // sample clock (8 kHz)
3     input wire reset,        // active-high reset
4     input wire signed [15:0] x_in, // input sample
5     output reg signed [15:0] y_out // output sample
6 );
7     parameter TAPS = 65; // default value 65
8
9     // shift register for storing delays shifting to the right
10    reg signed [15:0] x_delay [0:TAPS-1];
11
12    // store products from multiplier
13    wire signed [31:0] prod [0:TAPS-1];
14
15    integer i;
16    always @(posedge clk or posedge reset) begin
17        if (reset) begin // active-high reset
18            for (i=0 ; i<TAPS ; i=i+1) begin
19                x_delay[i] <= 16'sd0;
20            end
21        end
22        else begin
23            x_delay[0] <= x_in; // save
24            for (i=1 ; i<TAPS ; i=i+1) begin
25                x_delay[i] <= x_delay[i-1]; //shift down
26            end
27        end
28    end
29
30    // assign coefficients for MATLAB integers after x 32768
31
32    reg signed [15:0] coeff [0:TAPS-1];
33
34    integer k;
35    always @(*) begin
36        // default all zeros to avoid latches / unknowns
37        coeff[0]= -6;
38        coeff[1]= -0;
39        coeff[2]= 18;
40        coeff[3]= 0;
41        coeff[4]= -43;
42        coeff[5]= -0;
43        coeff[6]= 87;
44        coeff[7]= 0;
45        coeff[8]= -158;
46        coeff[9]= -0;
47        coeff[10]= 261;
48        coeff[11]= 0;
49        coeff[12]= 403;
50    end

```

Figure 9. Screenshot of the module of the Parameterizable FIR filter Module.

Part 4: Echo Machine

The `echo_machine.v` module generates an echo effect by combining the current audio sample with the delayed version of the same signal. The input signal is `x_in`, which is then passed through the shift register. The shift register stores the previous samples and outputs a `delayed_sample`. The delayed signal is then scaled by shifting it two bits (`>>>2`) which divides the amplitude by 4 ($2^2 = 4$) which reduces the echo intensity and prevents excessive signal amplification. The scaled delay signal `scaled_echo` is added to the original input signal using a built-in adder within the module with a wider bit-width of 19-bits to handle overflow during the addition. If reset is not active, the results are truncated back to 16-bits and stored in the output register “`y_out`” on the rising edge of the clock. If reset is activated, `y_out` is cleared to 0.

Flow Summary	
 <<Filter>>	
Flow Status	Successful - Fri Mar 6 20:06:09 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	echo_machine
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	500 / 32,070 (2 %)
Total registers	19
Total pins	34 / 457 (7 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 10. Screenshot of the Echo Machine module compilation report.

Table 3. Summary for Components for the Echo Machine Module in PreLab 3.

Feature	Quantity
Total number of logic elements	500
Total number of registers	19
Total number of pins	34
Max number of logic elements that can fit on FPGA	32,070

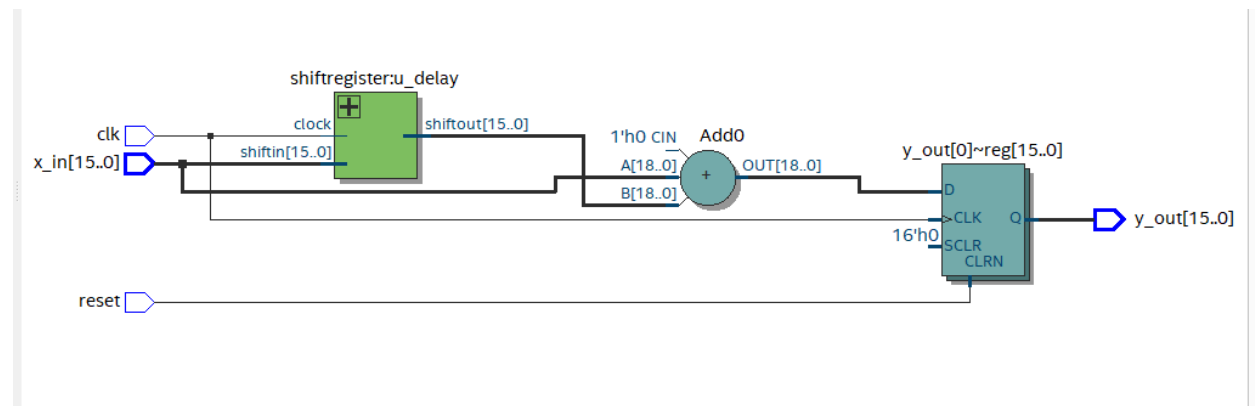
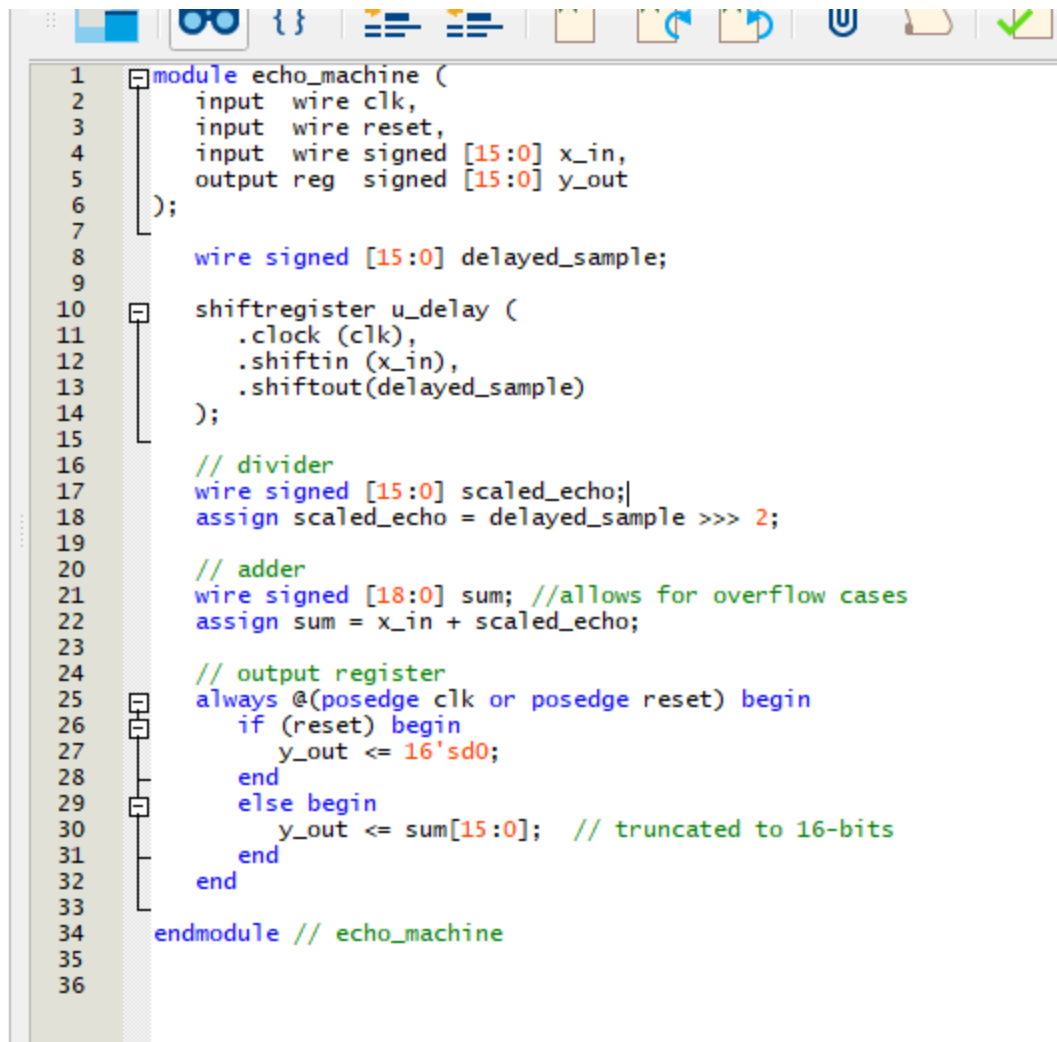


Figure 11. Screenshot of the implemented Echo Machine Module.



```
1 module echo_machine (  
2     input wire clk,  
3     input wire reset,  
4     input wire signed [15:0] x_in,  
5     output reg signed [15:0] y_out  
6 );  
7  
8     wire signed [15:0] delayed_sample;  
9  
10    shiftregister u_delay (  
11        .clock (clk),  
12        .shiftin (x_in),  
13        .shiftout(delayed_sample)  
14    );  
15  
16    // divider  
17    wire signed [15:0] scaled_echo;  
18    assign scaled_echo = delayed_sample >>> 2;  
19  
20    // adder  
21    wire signed [18:0] sum; //allows for overflow cases  
22    assign sum = x_in + scaled_echo;  
23  
24    // output register  
25    always @(posedge clk or posedge reset) begin  
26        if (reset) begin  
27            y_out <= 16'sd0;  
28        end  
29        else begin  
30            y_out <= sum[15:0]; // truncated to 16-bits  
31        end  
32    end  
33  
34 endmodule // echo_machine  
35  
36
```

Figure 12. Screenshot of the module of the Echo Machine Module.

Part 5: DSP Subsystem

The DSP subsystem module is used to instantiate the `fir_filter.v`, `echo_machine.v` and the `mux3_1.v` modules which connect it to the top-level module `lab3.v`. The inputs received in this module contain all the signals required by the three instantiated modules. The input signal is sent in parallel to the FIR filter, the echo machine and directly to the multiplexer as the original signal. The 3-to-1 multiplexer chooses between the original, filtered, or echoed signal based on the selector input. The selected signal is assigned to the `output_sample` and returned to the top-level module as the audio output.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Mar 6 19:57:12 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	dsp_subsystem
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	763 / 32,070 (2 %)
Total registers	1191
Total pins	36 / 457 (8 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	64 / 87 (74 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 13. Screenshot of the DSP Subsystem module compilation report.

Table 4. Summary for Components for the DSP Subsystem Module in PreLab 3.

Feature	Quantity
Total number of logic elements	763
Total number of registers	1191
Total number of pins	36
Max number of logic elements that can fit on FPGA	32,070

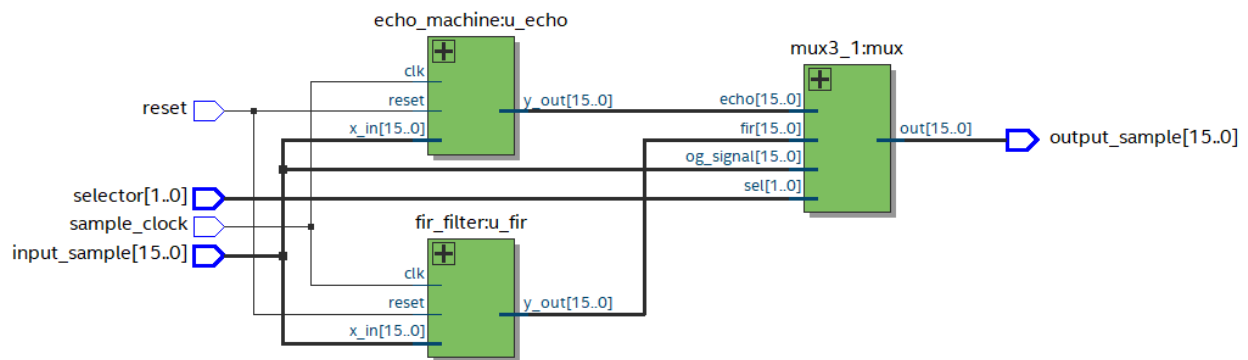


Figure 14. Screenshot of the implemented DSP Subsystem Module.

```

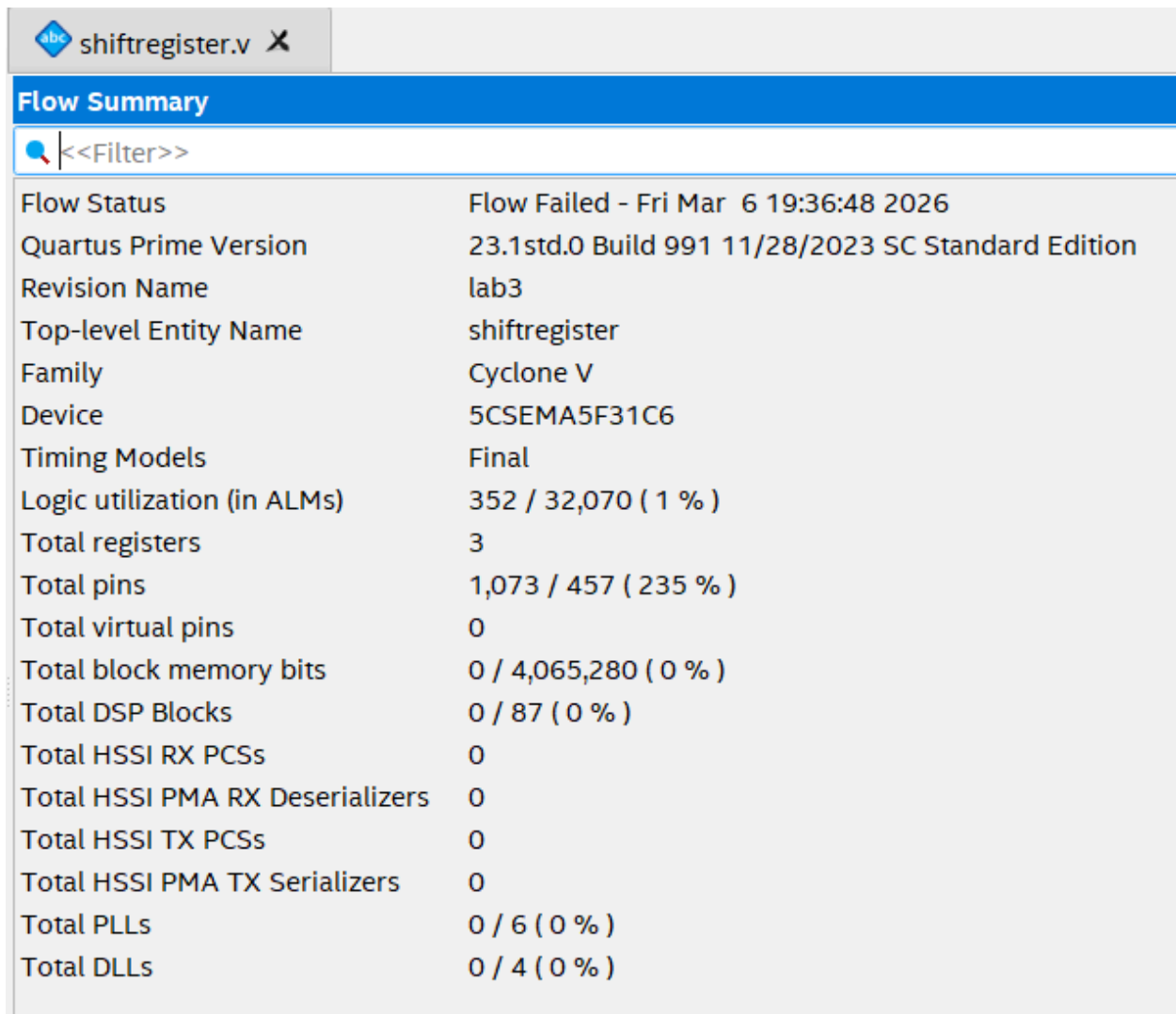
1  module dsp_subsystem (
2      input wire sample_clock,
3      input wire reset,
4      input wire [1:0] selector,
5      input wire signed [15:0] input_sample,
6      output signed [15:0] output_sample);
7
8      wire signed [15:0] y_fir;
9      wire signed [15:0] y_echo;
10     wire signed [15:0] mux_out;
11
12     // FIR filter
13     fir_filter u_fir (
14         .clk (sample_clock),
15         .reset (reset),
16         .x_in (input_sample),
17         .y_out(y_fir)
18     );
19     defparam u_fir.TAPS = 65;
20
21     // echo machine
22     echo_machine u_echo (
23         .clk (sample_clock),
24         .reset (reset),
25         .x_in (input_sample),
26         .y_out (y_echo)
27     );
28
29     // 3-to-1 multiplexer
30     mux3_1 mux (
31         .og_signal(input_sample),
32         .fir(y_fir),
33         .echo(y_echo),
34         .sel(selector),
35         .out(mux_out)
36     );
37
38     assign output_sample = mux_out;
39 endmodule // dsp_subsystem
40

```

Figure 15. Screenshot of the module of the DSP Subsystem Module.

Part 6: Generated modules Shift register and Multiplier

The shift-register and multiplier used within echo machine and the FIR modules, respectively, were generated by Quartus Prime using the IP Catalog > Library. The multiplier used Library>Basic Function>Arithmetic, whereas the shift register used Library>On Chip Memory>Shift Register (RAM-based). The multiplier used 2 inputs at 16-bits and created a product of 32-bits rather than the recommendation on the lab manual. This was done in order to ensure there was no data loss after using the multiplier since it still needed to be summed up. Therefore this method was used instead as it results in a much more accurate final result. The shift register uses the inputs to create a stored delayed list. In this case, the number of taps used for the shift-register is based on the last 2 digits of the student number, where 65 is used in this case. The tap distance is also set to 8, similarly to the calculation obtained in Part 1. The total bits required is based on the number of taps multiplied by the number of bits per tap, resulting in the final output $65 \times 16 = 1040$.



Flow Status	Flow Failed - Fri Mar 6 19:36:48 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	shiftregister
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	352 / 32,070 (1 %)
Total registers	3
Total pins	1,073 / 457 (235 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 16. Screenshot of the Shift Register module compilation report.

The shiftregister.v module failed because shiftregister was treated as the top-level module and the total number of pins would have been all the inputs and outputs combined. This would be an excessive amount over the amount of pins the board has hence the failure that occurred. The taps itself would have required 1040 pins which already exceed the total 457 pins. There is nothing wrong with the shiftregister.v module or the implementation of the code, but since it was set as the top-level module, the 1040-bit taps bus was also treated as external FPGA pins.

Table 5. Summary for Components for the Shift Register Module in PreLab 3.

Feature	Quantity
Total number of logic elements	352
Total number of registers	3
Total number of pins	1073
Max number of logic elements that can fit on FPGA	32,070

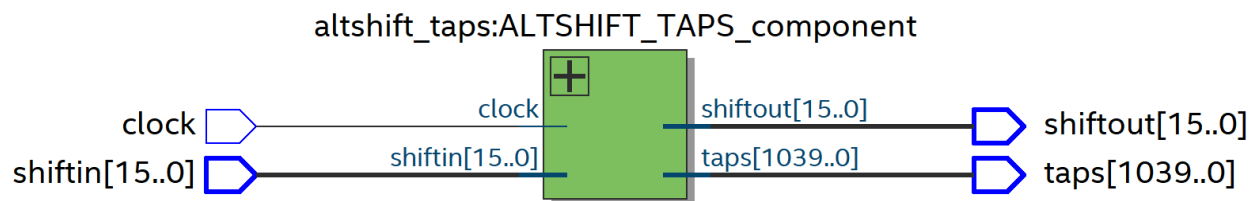


Figure 17. Screenshot of the implemented Shift Register Module.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Mar 6 19:44:48 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	multiplier
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	1 / 32,070 (< 1 %)
Total registers	0
Total pins	64 / 457 (14 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	1 / 87 (1 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 19. Screenshot of the Multiplier module compilation report.

Table 6. Summary for Components for the Multiplier Module in PreLab 3.

Feature	Quantity
Total number of logic elements	1
Total number of registers	0
Total number of pins	64
Max number of logic elements that can fit on FPGA	32,070

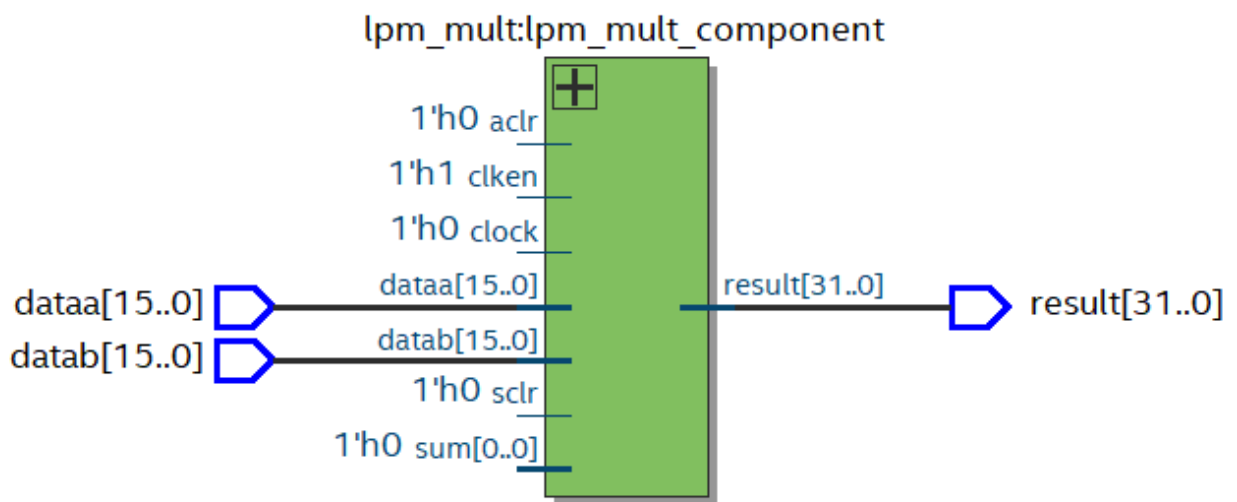


Figure 20. Screenshot of the implemented Multiplier Module


```

37 // synopsys translate_off
38 `timescale 1 ps / 1 ps
39 // synopsys translate_on
40 module multiplier (
41     dataa,
42     datab,
43     result);
44
45     input [15:0] dataa;
46     input [15:0] datab;
47     output [31:0] result;
48
49     wire [31:0] sub_wire0;
50     wire [31:0] result = sub_wire0[31:0];
51
52     lpm_mult lpm_mult_component (
53         .dataa (dataa),
54         .datab (datab),
55         .result (sub_wire0),
56         .aclr (1'b0),
57         .clken (1'b1),
58         .clock (1'b0),
59         .sclr (1'b0),
60         .sum (1'b0));
61
62     defparam
63         lpm_mult_component.lpm_hint = "MAXIMIZE_SPEED=5",
64         lpm_mult_component.lpm_representation = "SIGNED",
65         lpm_mult_component.lpm_type = "LPM_MULT",
66         lpm_mult_component.lpm_widtha = 16,
67         lpm_mult_component.lpm_widthb = 16,
68         lpm_mult_component.lpm_widthp = 32;
69
70 endmodule

```

Figure 21. Screenshot of generated multiplier.v module

Part 7: Top Level Module

The top level module was provided and no alterations were made. The top level module is able to interact with the rest of the modules we created without changing it since it instantiated dsp_subsystem.v. This is possible since the two DSP and multiplexer modules are instantiated within it.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri Mar 6 19:31:09 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab3
Top-level Entity Name	lab3
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	828 / 32,070 (3 %)
Total registers	1300
Total pins	13 / 457 (3 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	64 / 87 (74 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 22. Screenshot of the Top Level module compilation report.

Table 7. Summary for Components for the Top Level Module in PreLab 3.

Feature	Quantity
Total number of logic elements	828
Total number of registers	1300
Total number of pins	13
Max number of logic elements that can fit on FPGA	32,070

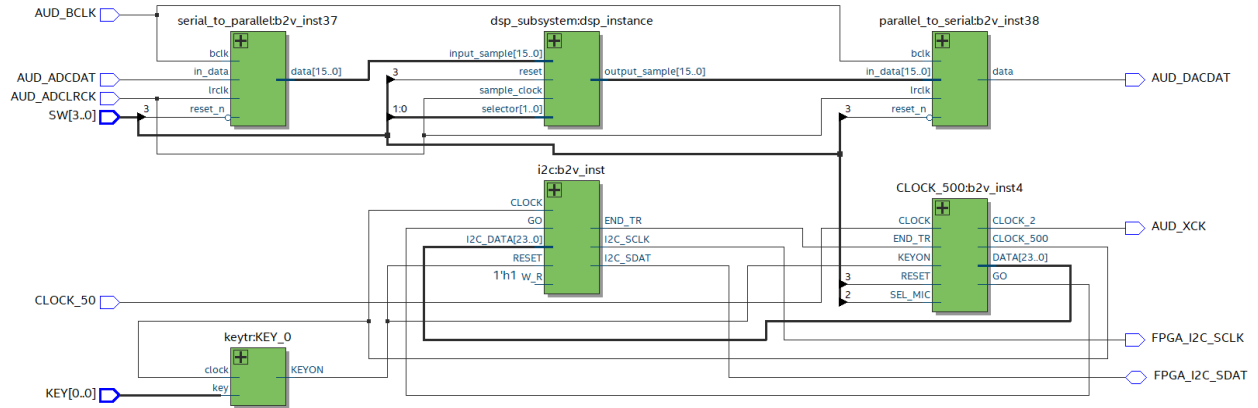


Figure 23. Screenshot of the implemented Top Level Module.

```

1  // This is the top-level design file for lab 3
2  // It instantiates all other modules in the system,
3  // including your DSP subsystem
4  // You can browse through it and read comments to
5  // understand the purpose of each module
6  // Please note that the modules with the line "Copyright (c) 2005 by Terasic Technologies Inc."
7  // were designed by someone at the said company (which manufactures the boards).
8  // The person who wrote them either had to do this at short notice or did NOT have good Verilog
9  // and FPGA design skills. Therefore, you should NOT consider these Verilog code samples
10 // as good examples of how hardware should be designed using Verilog
11 // Also note that the fact that most wire names are capitalized has NO special meaning.
12
13 module lab3(
14     KEY,
15     AUD_ADCDAT,
16     AUD_BCLK,
17     AUD_ADCLRCK,
18     CLOCK_50,
19     SW,
20     FPGA_I2C_SCLK,
21     AUD_XCK,
22     AUD_DACDAT,
23     FPGA_I2C_SDAT
24 );
25
26 input [0:0] KEY;
27 input AUD_ADCDAT;
28 input AUD_BCLK;
29 input AUD_ADCLRCK;
30 input CLOCK_50;
31 input [3:0] SW;
32 output FPGA_I2C_SCLK;
33 output AUD_XCK;
34 output AUD_DACDAT;
35 inout FPGA_I2C_SDAT;
36
37 wire CLOCK_1MHZ;
38 wire END_TR;
39 wire KEYON;
40 wire [1:0] sel;
41 wire sound;
42 wire XCK;
43 wire GO;
44 wire [23:0] CONFIGURATION_DATA;
45 wire [15:0] DATA_FROM_CODEC_TO_DSP;
46 wire [15:0] DATA_FROM_DSP_TO_CODEC;
47 wire SELECT_MICROPHONE;
48 wire reset_n;
49 wire reset;
50
51 // Selection line for the multiplexer comes from the switches 1 and 0

```

Figure 24. Screenshot of the module of the Top Level Module.

Code

Lab Tutorial 3

MATLAB Digital Filter

```
% Read the .wav file (replace file_name by your group's file)
% Variable x stores the wave and fs stores the sampling rate
[x, fs] = audioread('L01_Group6.wav');

% Perform FFT on the original signal to determine the frequency of the 'noise'
L = length(x);
NFFT = 2^nextpow2(L);

X = fft(x,NFFT)/fs;

% Show the sampling rate
display(fs)
% We know the sampling rate is 8000Hz.

% We need now to plot our FFT to find the source of the noise.
% Plot single-sided amplitude spectrum
f = fs/2*linspace(0,1,NFFT/2+1);
figure;
plot(f,2*abs(X(1:NFFT/2+1)));
title("Original Signal")
xlabel("Frequency (Hz)")
ylabel("|X(f)|")

% Reading the FFT we realize that the frequency we want to remove is 2000Hz.
% (Hint: our noise is a pure sine wave)
% Now specify the frequency you want to eliminate by setting:
% Hint: fkill is always in the range of 0 { 1, and is normalized to frequency of fs/2.
f_noise = 2000;
fkill = f_noise/(fs/2);

% Determine the coefficients of the FIR filter that will remove that frequency.
% Start off the following blank with the value 4, to numbers larger than 160.
filter_len = 64;

% Note: the following filter only works with EVEN numbers.
coeff = firgr(filter_len, ...
    [0,fkill-0.1, fkill, fkill+0.1, 1], ...
```

```
[1,1,0,1,1], ...  
{'n','n','s','n','n'});
```

```
% Plot the filter  
% Plot the frequency response of the designed filter to verify that it satisfies the  
% Requirements  
figure;  
freqz(coeff,1);  
title("Frequency Response of FIR Filter");
```

```
% You should try different filter lengths in the firr command and find out which one is  
% the best. Filter length of 4 is terrible. Ideally, your filter should only filter out the  
% noise while passing all other signals. Try increasing your filter length until you can  
% achieve an adequate result.  
% Be sure to plot (with freqz()) each time you create a new filter. If the filter length  
% is too large, the filter will "blow up". If you are unsure whether your filter has  
% blown up or not, seek help from a TA.  
coeff*32768
```

```
% Save these coefficients in a text file, You will need them when coding the FIR filter.  
fid = fopen('coefficients.txt','w');
```

```
% If you make a typing error with the following for-end block, you need to start from the "for" line  
again.
```

```
for i = 1:length(coeff)  
    fprintf(fid,'coeff[%3.0f]=%10.0f;\n',i-1,32768*coeff(i));  
end  
fclose(fid);
```

```
% Filter the input signal x(t) using the designed FIR filter to get y(t).  
y = filtfilt(coeff,1,x);  
% Perform FFT on the filtered signal to observe the absence of frequency of the "noise".  
Y = fft(y,NFFT)/L;
```

```
% Play the unfiltered sound (your system must have a working speaker or headphone)  
% Multiply by 3 to make the volume 3 times louder.  
sound(3*x,fs);  
% Pause 5 seconds. (this is only necessary if you run these commands as a script)  
pause(5);  
% Play the filtered sound. If cannot hear, try to change "3" to a larger integer  
% such as "10" in the following sound function.  
% Be careful of the possible damage to your hearing!  
sound(3*y,fs);
```

% The secret code for your group is JTAC03.

% Create two plots to compare

figure;

subplot(2,1,1);

% The first plot shows the FFT of the original signal.

plot(f,2*abs(X(1:NFFT/2+1)));

xlabel('Frequency (Hz)');

ylabel('|X(f)|');

title("FFT of Original Signal")

% The second plot shows the FFT of the filtered signal.

subplot(2,1,2);

plot(f, 2*abs(Y(1:NFFT/2+1)));

xlabel('Frequency(Hz)');

ylabel('|Y(f)|');

title("FFT of Filtered Signal")

% Write the filtered audio file to disk. If cannot hear anything when play the

% following .wav file, try to change "y" to "3*y", or with a larger integer,

% in the following function.

% Be careful of the possible damage to your hearing when play the .wav file!

audiowrite('L01_G06_Filtered.wav',y,fs);

Prelab 3

3-to-1 Multiplexer

```
module mux3_1(input signed [15:0] og_signal, fir, echo,
              input [1:0] sel,
              output reg signed [15:0] out);

    always @(*) begin
        case (sel)
            2'b00: out = og_signal;
            2'b01: out = fir;
            2'b10: out = echo;
            default: out = og_signal;
        endcase
    end
endmodule // mux3_1
```


Parameterizable FIR filter

```
module fir_filter(
    input wire clk,      // sample clock (8 kHz)
    input wire reset,    // active-high reset
    input wire signed [15:0] x_in,  // input sample
    output reg signed [15:0] y_out   // output sample
);

    parameter TAPS = 65; // default value 65

    // shift register for storing delays shifting to the right
    reg signed [15:0] x_delay [0:TAPS-1];

    // store products from multiplier
    wire signed [31:0] prod [0:TAPS-1];

    integer i;
    always @(posedge clk or posedge reset) begin
        if (reset) begin // active-high reset
            for (i=0 ; i<TAPS ; i=i+1) begin
                x_delay[i] <= 16'sd0;
            end
        end
        else begin
            x_delay[0] <= x_in; // save
            for (i=1 ; i<TAPS ; i=i+1) begin
                x_delay[i] <= x_delay[i-1]; //shift down
            end
        end
    end

    // assign coefficients for MATLAB integers after x 32768
    reg signed [15:0] coeff [0:TAPS-1];

    integer k;
    always @(*) begin
        // default all zeros to avoid latches / unknowns
        coeff[0]=    -6;
        coeff[1]=    -0;
        coeff[2]=    18;
        coeff[3]=     0;
        coeff[4]=   -43;
        coeff[5]=    -0;
        coeff[6]=    87;
```

```
coeff[7]= 0;
coeff[8]= -158;
coeff[9]= -0;
coeff[10]= 261;
coeff[11]= 0;
coeff[12]= -403;
coeff[13]= -0;
coeff[14]= 585;
coeff[15]= 0;
coeff[16]= -807;
coeff[17]= -0;
coeff[18]= 1061;
coeff[19]= 0;
coeff[20]= -1338;
coeff[21]= -0;
coeff[22]= 1620;
coeff[23]= 0;
coeff[24]= -1890;
coeff[25]= -0;
coeff[26]= 2127;
coeff[27]= 0;
coeff[28]= -2313;
coeff[29]= -0;
coeff[30]= 2431;
coeff[31]= 0;
coeff[32]= 30296;
coeff[33]= 0;
coeff[34]= 2431;
coeff[35]= -0;
coeff[36]= -2313;
coeff[37]= 0;
coeff[38]= 2127;
coeff[39]= -0;
coeff[40]= -1890;
coeff[41]= 0;
coeff[42]= 1620;
coeff[43]= -0;
coeff[44]= -1338;
coeff[45]= 0;
coeff[46]= 1061;
coeff[47]= -0;
coeff[48]= -807;
coeff[49]= 0;
coeff[50]= 585;
```

```

coeff[51]=    -0;
coeff[52]=   -403;
coeff[53]=     0;
coeff[54]=    261;
coeff[55]=    -0;
coeff[56]=   -158;
coeff[57]=     0;
coeff[58]=    87;
coeff[59]=    -0;
coeff[60]=   -43;
coeff[61]=     0;
coeff[62]=    18;
coeff[63]=    -0;
coeff[64]=    -6;

end

// 16x16 signed to 32-bit signed product of coefficients multiplied by 32768
genvar t;
generate
    for (t = 0; t < TAPS; t = t + 1) begin : GEN_MUL
        multiplier u_mult (
            .dataa(x_delay[t]), // 16-bit signed
            .datab(coeff[t]),   // 16-bit signed
            .result(prod[t])    // 32-bit signed
        );
    end
endgenerate

// summing all true coefficients together (adder built in)
reg signed [31:0] sum;
integer s;
always @(*) begin
    sum = 32'sd0;
    for (s=0 ; s<TAPS ; s=s+1)
        // summing together all products
        sum = sum + prod[s];
end

// divide by 2^15 (32768)
wire signed [31:0] sum_scaled;
assign sum_scaled = sum >>> 15;

// output register per clock cycle
always @(posedge clk or posedge reset) begin

```

```
        if (reset) begin
            y_out <= 16'sd0;
        end
        else begin
            y_out <= sum_scaled[15:0]; // summed scaled sum truncated to 16 bits
        end
    end
end

endmodule // fir_filter
```

Echo Machine

```
module echo_machine (
    input wire clk,
    input wire reset,
    input wire signed [15:0] x_in,
    output reg signed [15:0] y_out
);

    wire signed [15:0] delayed_sample;

    shiftregister u_delay (
        .clock (clk),
        .shiftin (x_in),
        .shiftout(delayed_sample)
    );

    // divider
    wire signed [15:0] scaled_echo;
    assign scaled_echo = delayed_sample >>> 2;

    // adder
    wire signed [18:0] sum; //allows for overflow cases
    assign sum = x_in + scaled_echo;

    // output register
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            y_out <= 16'sd0;
        end
        else begin
            y_out <= sum[15:0]; // truncated to 16-bits
        end
    end

endmodule // echo_machine
```

DSP Subsystem

```
module dsp_subsystem (  
    input wire sample_clock,  
    input wire reset,  
    input wire [1:0] selector,  
    input wire signed [15:0] input_sample,  
    output signed [15:0] output_sample);  
  
    wire signed [15:0] y_fir;  
    wire signed [15:0] y_echo;  
    wire signed [15:0] mux_out;  
  
    // FIR filter  
    fir_filter u_fir (  
        .clk (sample_clock),  
        .reset (reset),  
        .x_in (input_sample),  
        .y_out(y_fir)  
    );  
    defparam u_fir.TAPS = 65;  
  
    // echo machine  
    echo_machine u_echo (  
        .clk (sample_clock),  
        .reset (reset),  
        .x_in (input_sample),  
        .y_out (y_echo)  
    );  
  
    // 3-to-1 multiplexer  
    mux3_1 mux (  
        .og_signal(input_sample),  
        .fir(y_fir),  
        .echo(y_echo),  
        .sel(selector),  
        .out(mux_out)  
    );  
  
    assign output_sample = mux_out;  
endmodule // dsp_subsystem
```

Multiplier

```
// synopsys translate_off
`timescale 1 ps / 1 ps
// synopsys translate_on
module multiplier (
    dataa,
    datab,
    result);

    input  [15:0] dataa;
    input  [15:0] datab;
    output [31:0] result;

    wire [31:0] sub_wire0;
    wire [31:0] result = sub_wire0[31:0];

    lpm_mult      lpm_mult_component (
        .dataa (dataa),
        .datab (datab),
        .result (sub_wire0),
        .aclr (1'b0),
        .clken (1'b1),
        .clock (1'b0),
        .sclr (1'b0),
        .sum (1'b0));

    defparam
        lpm_mult_component.lpm_hint = "MAXIMIZE_SPEED=5",
        lpm_mult_component.lpm_representation = "SIGNED",
        lpm_mult_component.lpm_type = "LPM_MULT",
        lpm_mult_component.lpm_widtha = 16,
        lpm_mult_component.lpm_widthb = 16,
        lpm_mult_component.lpm_widthp = 32;

endmodule
```

Shift Register

```
`timescale 1 ps / 1 ps
// synopsys translate_on
module shiftregister (
    clock,
    shiftin,
    shiftout,
    taps);

    input    clock;
    input  [15:0] shiftin;
    output [15:0] shiftout;
    output [1039:0] taps;

    wire [15:0] sub_wire0;
    wire [1039:0] sub_wire1;
    wire [15:0] shiftout = sub_wire0[15:0];
    wire [1039:0] taps = sub_wire1[1039:0];

    altshift_taps  ALTSHIFT_TAPS_component (
        .clock (clock),
        .shiftin (shiftin),
        .shiftout (sub_wire0),
        .taps (sub_wire1)
        // synopsys translate_off
        ,
        .aclr (),
        .clken (),
        .sclr ()
        // synopsys translate_on
    );

    defparam
        ALTSHIFT_TAPS_component.intended_device_family = "Cyclone V",
        ALTSHIFT_TAPS_component.lpm_hint = "RAM_BLOCK_TYPE=MLAB",
        ALTSHIFT_TAPS_component.lpm_type = "altshift_taps",
        ALTSHIFT_TAPS_component.number_of_taps = 65, //just change this here to #
7 for 07 student number
        ALTSHIFT_TAPS_component.tap_distance = 8,
        ALTSHIFT_TAPS_component.width = 16;

endmodule
```